

R - Adattáblák és Titanic: megoldókulcs

Megjegyzés

A cél nem csak az eredmény, hanem az indexelés, szűrés, sor- és oszlopválasztás, rendezés és csoportosítás logikájának megértése. A Titanic-fájlban több életkor hiányzik; ezért az átlagoknál külön figyelni kell a `na.rm=T` használatára.

Adattáblák létrehozása és összekapcsolása

```
nev=c("András","Béla","Ilona","Judit","Zoltán")
```

Létrehoz egy 5 elemű karaktervektort nevekkal. Ezt később az adattábla egyik oszlopaként használjuk.

```
neme=factor(c(0,0,1,1,0))
```

A 0 és 1 értékekből faktort készít. A factor kategóriaváltozó: itt két lehetséges szint van, a 0 és az 1. Nem egyszerű számként, hanem kategóriaként kezeli őket.

```
magassag=c(187,192,167,172,183)
```

Létrehoz egy 5 elemű numerikus vektort a magasságokkal.

```
tabla=data.frame(nev,neme,magassag)
```

A három azonos hosszúságú vektorból adattáblát készít. Az oszlopok neve `nev`, `neme` és `magassag`; minden személy egy sort kap.

```
tabla
```

Kiírja a teljes tabla adattáblát a konzolra.

```
tabla2=data.frame(iq=c(101,132,114),eletkor=c(24,33,19))
```

Létrehoz egy második adattáblát két oszloppal és 3 sorral: `iq` és `eletkor`.

```
tabla2
```

Kiírja a `tabla2` adattáblát.

```
tabla3=data.frame(tabla,tabla2)
```

Ez HIBÁT ad: a `tabla` 5 soros, a `tabla2` pedig 3 soros. Egymás mellé csak akkor tehetők így az oszlopok, ha a sorszámok összeegyeztethetők. Az 5 és 3 itt nem az.

```
tabla2=data.frame(iq=c(101,132,114,98,120),eletkor=c(24,33,19,45,37))
```

Újradefiniálja a `tabla2`-t, most már 5 sorral. Így ugyanannyi sora van, mint a `tabla` objektumnak.

```
tabla3=data.frame(tabla,tabla2)
```

Most már sikerül egymás mellé tenni a két adattábla oszlopait, mert mindkettő 5 soros.

```
tabla3
```

Kiírja az 5 soros, 5 oszlopos tabla3 adattáblát.

```
tabla4=data.frame(tabla3,c("Budapest", "Győr", "Pápa", "Debrecen", "Budapest"))
```

A tabla3 mellé hozzáad egy új, 5 elemű szöveges oszlopot a lakhelyekkel. Az új oszlop neve ekkor még automatikusan generált, ezért utána átnevezzük.

```
tabla4
```

Kiírja a tabla4 adattáblát, benne az új lakhely-oszloppal.

```
rownames(tabla4)
```

Lekéri a sorneveket. Alapból ezek általában "1", "2", "3", ... alakúak.

```
colnames(tabla4)[6]="lakhely"
```

A 6. oszlop nevét lakhely-re változtatja. A colnames(tabla4) az összes oszlopnevet adná vissza, a [6] pedig ezek közül a hatodikat választja ki.

```
tabla4
```

Újra kiírja a táblát, így már látható a lakhely oszlopnév.

Adattábla indexelése

```
tabla4[4,3]
```

A 4. sor 3. oszlopának egyetlen értékét választja ki. Itt ez Judit magassága: 172.

```
tabla4[c(1,2,5),]
```

Az 1., 2. és 5. sort választja ki, minden oszloppal. A vessző után nincs oszlopindex, ezért minden oszlop marad.

```
tabla4[-1,c(4,5)]
```

Az első sort kihagyja, és csak a 4. és 5. oszlopot tartja meg. A negatív sorindex azt jelenti: ezt a sort ne kérd.

```
tabla4
```

Kiírja az eredeti tabla4-et. Az előző indexelések nem módosították, csak kiválasztottak belőle részeket.

```
rownames(tabla4)
```

Ismét lekéri a sorneveket. Ezen a ponton még számozott sornevek vannak.

```
tabla4$nev
```

A \$ jellel név szerint választunk ki egy oszlopot. Ez a nev oszlop összes elemét adja vissza vektorként.

```
tabla4$magassag
```

A magassag oszlopot adja vissza vektorként.

```
rownames(tabla4)=tabla4$nev
```

A sorneveket a nev oszlop értékeire állítja. Ettől kezdve a sorokra például "Judit" vagy "András" névvel is hivatkozhatunk.

```
tabla4
```

Kírja a táblát az új, nevekből álló sornevekkel. A nev oszlop ettől még továbbra is benne van.

```
tabla5=tabla4[, -1]
```

Létrehozza a tabla5-öt a tabla4 első oszlopa nélkül. A sornevek megmaradnak, ezért a nev oszlop már feleslegessé válhat.

```
tabla5
```

Kírja a tabla5 adattáblát.

```
tabla5[4,]
```

A 4. sort választja ki minden oszloppal. A sor neve Judit.

```
tabla5["Judit",]
```

Ugyanazt a Judit-sort választja ki, de sorszám helyett a sor nevét használja. Ez az egyik előnye a beszédes sorneveknek.

```
tabla5["András", "magassag"]
```

Az András nevű sorból a magassag oszlop értékét kéri le. Eredménye 187.

```
tabla5["András", c("eletkor", "lakhely")]
```

András sorából egyszerre két oszlopot választ ki: életkor és lakhely. Eredmény: 24 és Budapest.

```
tabla5$iq
```

Az iq oszlopot adja vissza vektorként a \$ operátorral.

```
tabla5[, "iq"]
```

Szintén az iq oszlopot adja vissza. A két forma itt ugyanazt eredményezi: tabla5\$iq és tabla5[, "iq"].

```
tabla5["Judit",]
```

Ismét Judit teljes sorát választja ki név alapján.

Titanic CSV beolvasása

Megjegyzés

A titanic.csv fájlt tedd ugyanabba a mappába, ahol az R-script van. A feladat így internet nélkül is működik. A fájl kettősponttal (:) tagolt, ezért sep=":" kell.

```
read.table("titanic.csv", header=T, sep=": ")
```

Beolvassa a helyi titanic.csv fájlt. A header=T azt jelenti, hogy az első sor oszlopneveket tartalmaz; a sep=":" pedig azt, hogy a mezőket kettőspont választja el.

```
titanic=data.frame(read.table("titanic.csv", header=T, sep=": "))
```

Beolvassa a fájlt, majd az eredményt titanic néven adattáblaként eltárolja. A data.frame() itt lényegében megerősíti, hogy adattáblaként dolgozunk vele.

```
View(titanic)
```

Táblázatos nézetben megnyitja a teljes titanic adattáblát az RStudio adatnézőjében. A View() megjelenít, de nem módosít adatot.

Megjegyzés

Az eredeti jegyzetben titanic[1:80,] szerepelt, de a most használt Titanic-fájlban "Bing, Mr Lee" a 674. sorban van. Ezért itt 1:800-at használunk, hogy a név szerinti keresés működjön.

```
titanic2=titanic[1:800,]
```

A titanic első 800 sorát másolja a titanic2 adattáblába, minden oszloppal.

```
View(titanic2)
```

Megnyitja a titanic2 első 800 utasát táblázatos nézetben.

```
rownames(titanic2)=titanic2$Name
```

A Name oszlopban lévő utasneveket beállítja sorneveknek. Ezután egy konkrét utasra a nevével is hivatkozhatunk.

```
titanic2=titanic2[, -1]
```

Eltávolítja az első, Name oszlopot, mert a nevek már sornevekként szerepelnek.

```
titanic2["Bing, Mr Lee",]
```

Név alapján kiválasztja Bing, Mr Lee teljes sorát. A fájlban: 3rd osztály, 32 éves, male, Survived=1, SexCode=0.

```
titanic2["Bing, mr Lee",]
```

Nem talál pontos egyezést, mert a sornevek kis- és nagybetű érzékenyek: "Mr" nem ugyanaz, mint "mr". Emiatt hiányzó (NA) sort kapunk.

```
titanic2["Bing, Mr Lee", "Age"]
```

Bing, Mr Lee sorából csak az Age oszlopot kéri le. Eredménye 32.

```
titanic
```

Kírja a teljes Titanic-adattáblát. Ez 1313 utast és 6 oszlopot tartalmaz.

Szűrés és átlagok a Titanic-adatokon

Kérdés és válasz

Melyik osztályon volt a legmagasabb az átlagéletkor? Válasz: A nyers mean() hívások NA-t adnak, mert több Age érték hiányzik. na.rm=T használatával az ismert életkorok átlaga: 1st $\approx 39,67$; 2nd $\approx 28,30$; 3rd $\approx 25,21$. Ezek közül az 1st a legnagyobb.

```
titanic$PClass[titanic$PClass=="1st"]
```

Először létrejön egy logikai feltétel: mely sorokban 1st a PClass? Ezután a PClass oszlopból csak ezeket az elemeket tartja meg. Az eredmény sok "1st" érték lesz.

```
titanic[titanic$PClass=="1st",]
```

Kiválasztja az összes első osztályú utast, minden oszlopukkal.

```
mean(titanic[titanic$PClass=="1st", "Age"])
```

Az első osztályú utasok Age értékeinek átlagát próbálja kiszámolni. A fájlban vannak hiányzó életkorok, ezért az eredmény alapból NA. Használd: mean(..., na.rm=T). Ekkor kb. 39,67.

```
mean(titanic[titanic$PClass=="2nd", "Age"])
```

A másodosztály ismert életkorainak átlaga na.rm=T mellett kb. 28,30. na.rm nélkül NA lesz a hiányzó értékek miatt.

```
mean(titanic[titanic$PClass=="3rd", "Age"])
```

A harmadosztály ismert életkorainak átlaga na.rm=T mellett kb. 25,21. na.rm nélkül NA lesz.

Kérdés és válasz

A másodosztályon utazó férfiaknak vagy nőknek magasabb az átlagéletkora? Válasz: A hiányzó életkorokat elhagyva: férfiak $\approx 28,91$ év, nők $\approx 27,39$ év. Ebben az adatállományban a másodosztályú férfiak ismert életkorának átlaga magasabb.

```
titanic[titanic$PClass=="2nd",]
```

Kiválasztja az összes másodosztályú utast.

```
titanic[titanic$PClass=="2nd" & titanic$Sex=="male",]
```

Két feltételt kapcsol össze &: az utas legyen 2nd osztályú ÉS male. Csak azok a sorok maradnak, amelyek mindkettőnek megfelelnek.

```
titanic[titanic$PClass=="2nd" & titanic$Sex=="male",]$Age
```

A másodosztályú férfiak sorainak kiválasztása után a \$Age csak az életkor-oszlopot adja vissza.

```
titanic[titanic$PClass=="2nd" & titanic$Sex=="male", "Age"]
```

Ugyanazt az Age-vektort adja vissza, mint az előző sor, csak közvetlen oszlopindexeléssel.

```
mean(titanic[titanic$PClass=="2nd" & titanic$Sex=="male", "Age"])
```

Átlagot számolna a másodosztályú férfiak életkorából, de na.rm nélkül NA. na.rm=T mellett kb. 28,91.

```
mean(titanic[titanic$PClass=="2nd" & titanic$Sex=="female", "Age"])
```

A másodosztályú nők életkorának átlaga. na.rm=T mellett kb. 27,39; na.rm nélkül NA.

Kérdés és válasz

A középkorosztály (40-60) átlagéletkora? Válasz: A feltétel valójában szigorúan 40 fölött és 60 alatt válogat: 40 és 60 nem kerül bele. A hiányzó értékeket elhagyva az átlag kb. 48,37 év.

```
titanic[titanic$Age>40 & titanic$Age<60,]
```

Azokat a sorokat választja ki, ahol Age nagyobb 40-nél ÉS kisebb 60-nál. A határértékek (40 és 60) kimaradnak. Hiányzó Age esetén a feltétel NA lehet.

```
titanic[titanic$Age>40 & titanic$Age<60, "Age"]
```

Ugyanez a szűrés, de csak az Age oszlopot adja vissza.

```
mean(titanic[titanic$Age>40 & titanic$Age<60, "Age"])
```

A kiválasztott életkorok átlagát számítaná. Mivel NA értékek is bekerülhetnek, az eredeti forma NA-t ad. na.rm=T mellett kb. 48,37.

Kérdés és válasz

Férfi vagy nő volt több a hajón? Válasz: A fájlban 851 male és 462 female sor van. Tehát férfiból van több. Itt nincs hiányzó Sex érték.

```
titanic[titanic$Sex=="male",]
```

Kiválasztja az összes férfi utas teljes sorát.

```
titanic[titanic$Sex=="male", "Sex"]
```

Csak a Sex oszlopot tartja meg a férfi utasokból, így egy "male" értékekből álló vektort kapunk.

```
length(titanic[titanic$Sex=="male", "Sex"])
```

Megszámolja a kiválasztott férfi Sex-értékeket. Eredmény: 851.

```
length(titanic[titanic$Sex=="female", "Sex"])
```

Megszámolja a női utasokat. Eredmény: 462.

```
length(titanic$Sex) - length(titanic[titanic$Sex=="male", "Sex"])
```

Az összes Sex-érték számából kivonja a férfiak számát. Mivel ebben a fájlban csak male/female van és nincs hiányzó Sex, az eredmény 462, vagyis a nők száma.

```
length(titanic)
```

FONTOS: adattáblánál a length() az oszlopok számát adja, nem a sorokét. A titanic 6 oszlopos, ezért az eredmény 6.

```
length(titanic$Name)
```

A Name oszlop egy vektor, ezért ennek hossza az utasok száma. Eredmény: 1313.

Sorbarendezés

```
titanic$Age[order(titanic$Age)]
```

Az order(titanic\$Age) megadja, milyen sorrendben kellene venni a pozíciókat ahhoz, hogy az életkor növekvő legyen. Az indexelés ezek szerint rendezi az Age-vektort. A hiányzó értékek alaphoz a végére kerülnek.

```
titanic[order(titanic$Age),]
```

A teljes adattáblát életkor szerint növekvő sorrendbe rendezi, miközben minden sor többi adata együtt marad az utassal.

```
titanic[order(titanic$Name),]
```

A teljes táblát név szerint rendezi. A szövegek rendezését a gép nyelvi/beállítási környezete is befolyásolhatja.

```
titanic[order(titanic$Sex, titanic$Age),]
```

Először Sex szerint rendez, azon belül Age szerint növekvően. Az order() több rendezési kulcsot is kaphat.

```
titanic[order(titanic$Sex, titanic$PClass, -titanic$Age),]
```

Először Sex, azon belül PClass szerint rendez, majd az életkort csökkenő sorrendben, mert -titanic\$Age szerepel. A mínusz numerikus oszlopnál megfordítja a rendezés irányát.

Csoportba rendezés cut() segítségével

Komment

25-39: fiatal; 40-59: középkorosztály; 60-71: idős

```
cut(c(12,15,8,23,19,4,43,29),breaks=c(4,18,30,43))
```

A számokat intervallumokba sorolja: (4,18], (18,30], (30,43]. Alapból a jobb határ benne van, a bal nincs. Emiatt maga a 4 nem kerül egyik csoportba sem, NA lesz.

```
cut(c(12,15,8,23,19,4,43,29),breaks=c(2,18,30,43))
```

Most az első intervallum (2,18], ezért a 4 is belefér. A breaks a csoporthatárokat adja meg.

```
cut(c(12,15,8,23,19,4,43,29),breaks=c(2,18,30,43),labels=c("kicsi","közepes","nagy"))
```

Ugyanazokat a csoportokat készíti, de saját neveket ad nekik: kicsi, közepes, nagy. Az eredmény faktor lesz ezekkel a kategóriákkal.

```
cut(titanic$Age,breaks=c(24,39,59,80),labels=c("fiatal","középkorosztály","idős"))
```

Minden életkort korcsoportba sorol: (24,39] = fiatal, (39,59] = középkorosztály, (59,80] = idős. A 24 éves és annál fiatalabb utasok, valamint a hiányzó életkorok NA csoportot kapnak.

```
korosztaly=cut(titanic$Age,breaks=c(24,39,59,80),labels=c("fiatal","középkorosztály","idős"))
```

Ugyanezt a korcsoport-faktort eltárolja korosztaly néven.

```
titanic3=data.frame(korosztaly,titanic)
```

Az új korosztaly változót a Titanic-adattábla elé teszi új oszlopként. Így egy kibővített adattáblát kapunk.

```
titanic3
```

Kiírja a kibővített adattáblát, benne a korosztaly oszloppal.

Kérdés és válasz

Az első osztályon utazó fiatalok száma Válasz: A fiatal kategória itt (24,39], tehát 25-39 éveseket jelent. Az 1st osztályú fiatalok száma ebben a fájlban 70.

```
titanic3[titanic3$korosztaly=="fiatal",]
```

Kiválasztja az összes fiatal korcsoportba sorolt utast.

```
titanic3[titanic3$korosztaly=="fiatal" & titanic3$PClass=="1st",]
```

Két feltétellel szűr: korosztaly legyen fiatal ÉS PClass legyen 1st. Így csak az első osztályú fiatalok sorai maradnak.

```
length(titanic3[titanic3$korosztaly=="fiatal" & titanic3$PClass=="1st","Age"])
```

A szűrt sorok Age oszlopának hosszát veszi, vagyis megszámlolja a találatokat. Eredmény: 70.